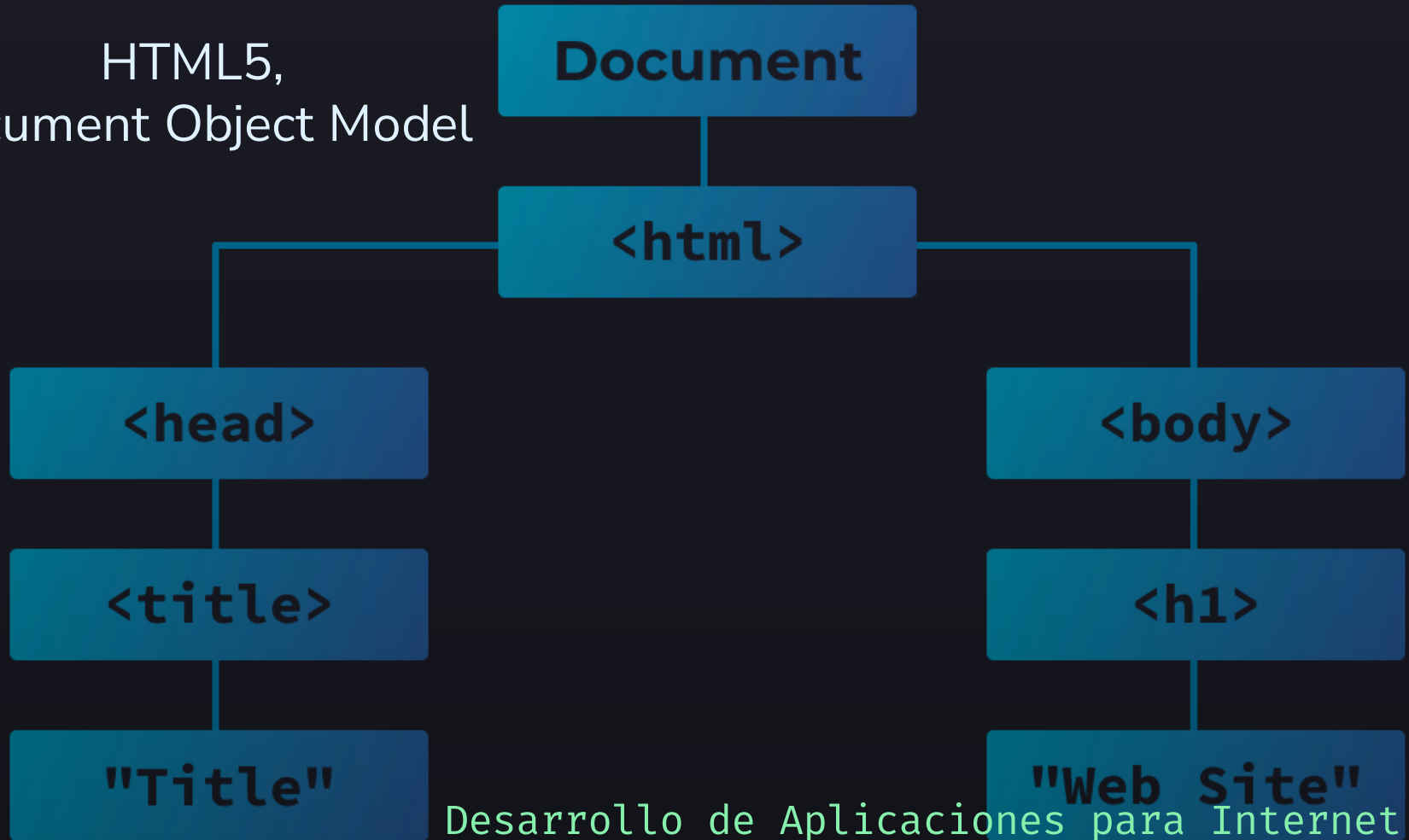
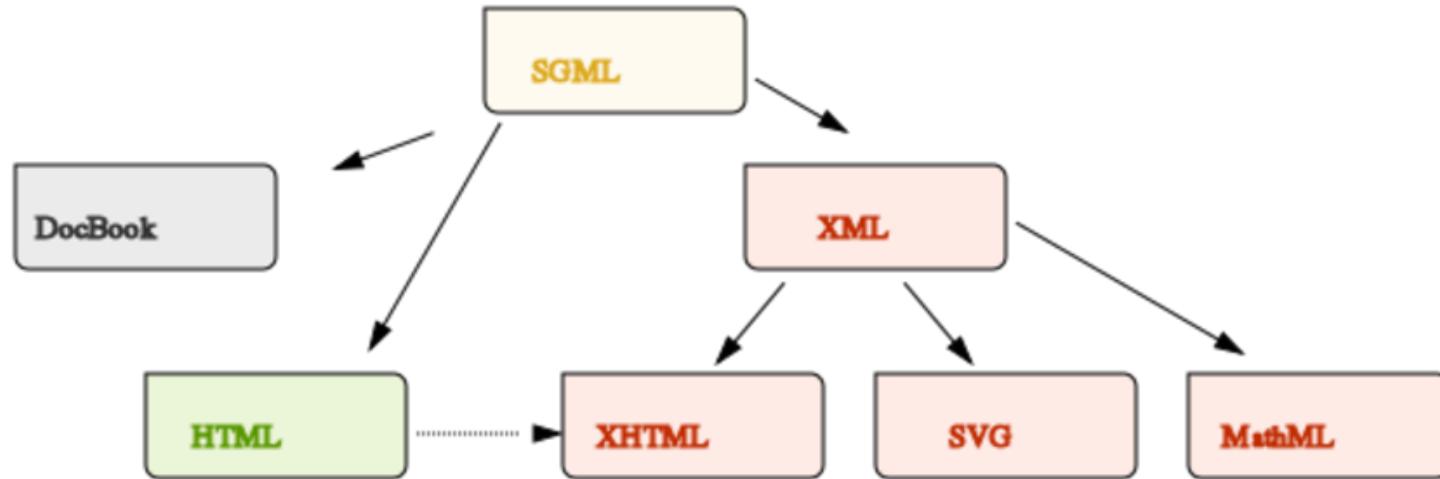


HTML5, Document Object Model



Lenguajes de marcas: SGML, HTML, XML, XHTML



Que es XML

Marcas legibles por
personas y máquinas,

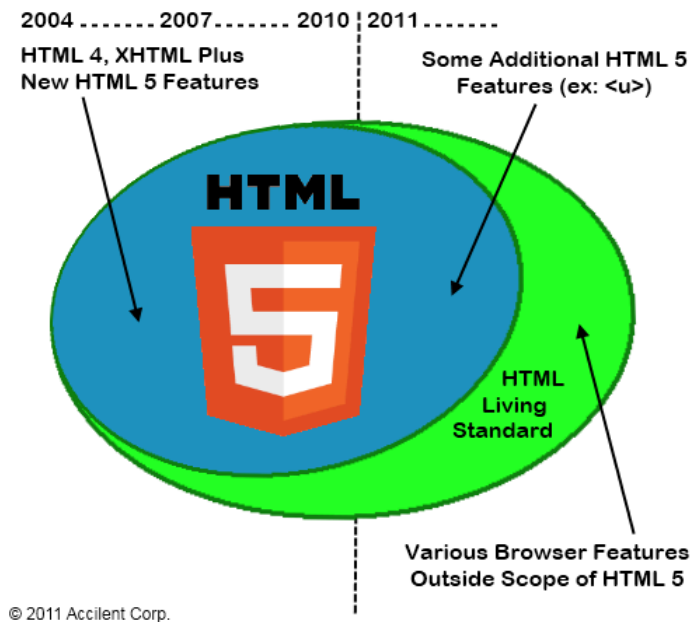
para dar significado a
partes de un texto.

```
<?xml version="1.0" encoding="UTF-8"?>
- <EmployeeData>
  - <employee id="34594">
    <firstName>Heather</firstName>
    <lastName>Banks</lastName>
    <hireDate>1/19/1998</hireDate>
    <deptCode>BB001</deptCode>
    <salary>72000</salary>
  </employee>
  - <employee id="34593">
    <firstName>Tina</firstName>
    <lastName>Young</lastName>
    <hireDate>4/1/2010</hireDate>
    <deptCode>BB001</deptCode>
    <salary>65000</salary>
  </employee>
</EmployeeData>
```

Diferencias HTML, XHTML

	HTML	XHTML
mayúsculas / minúsculas	<code><P></code> o <code><p></code>	<code><p></code> siempre minúscula
elementos con etiqueta de cierre	<code><p></code> ... <code><p></code>	<code><p></code> ... <code></p></code> <code><p></code> ... <code></p></code> siempre elementos cerrados
etiqueta con autocierre	<code></code>	<code></code> o con autocierre
atributos con comillas	<code><p id=4></code>	<code><p id="4"></code> siempre comillas en los atributos
atributos con valor	<code><input required></code>	<code><input required="true"/></code>

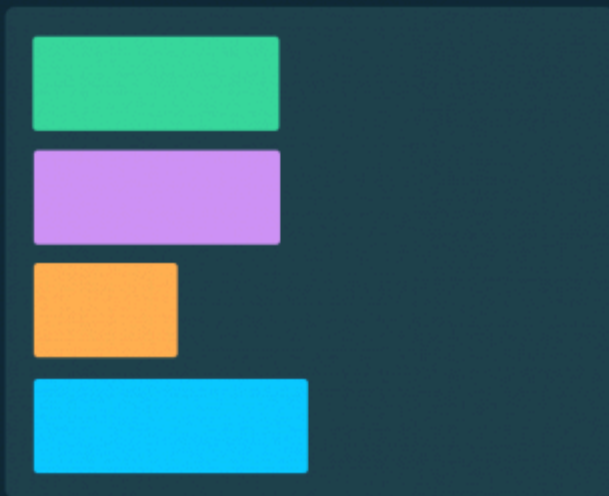
HTML5, living standar



Top 10 html elements to use in 2023

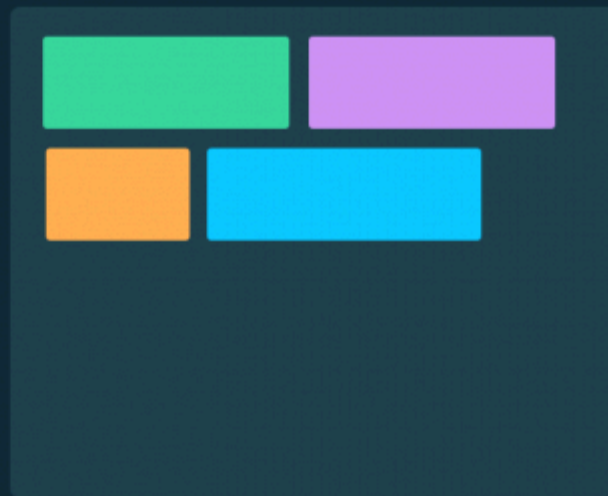
Los elementos de HTML son block o inline

BLOCK VS INLINE



display: block;

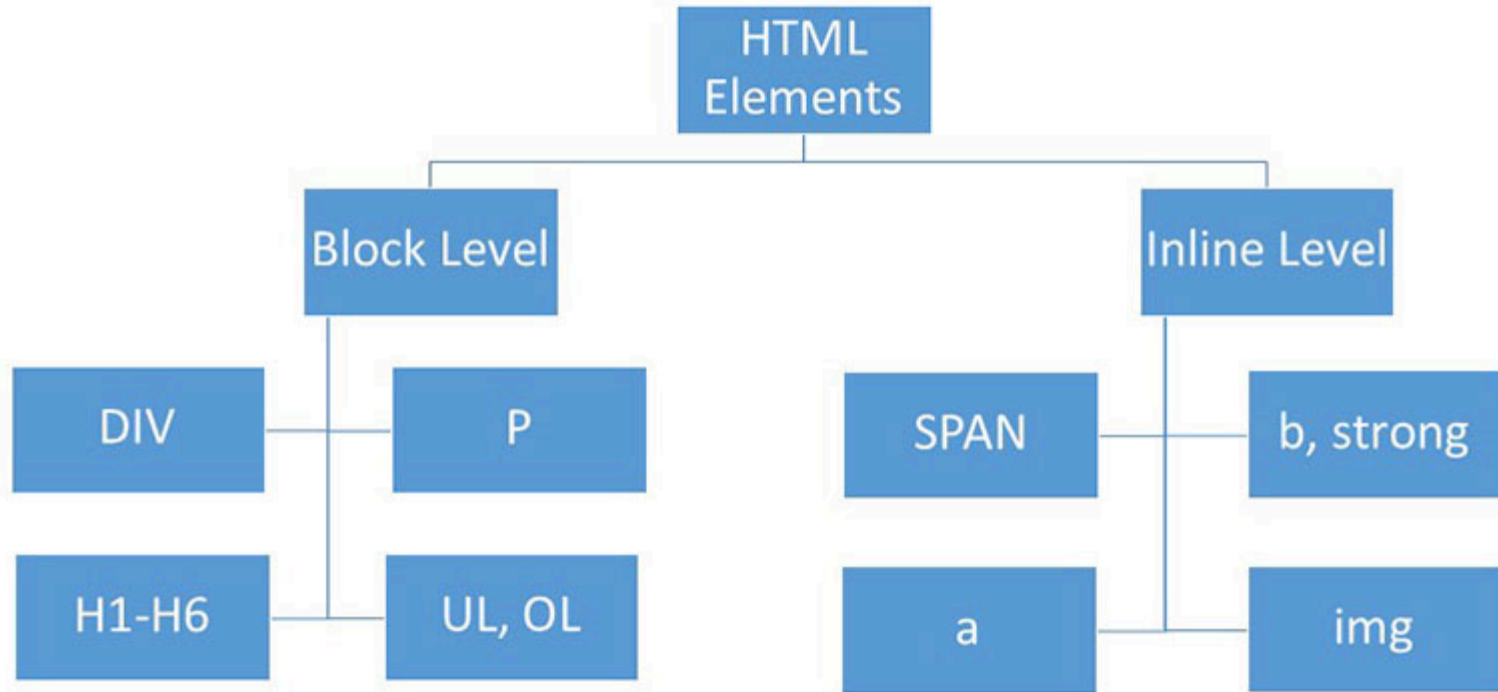
Block elements stack, regardless of their width.

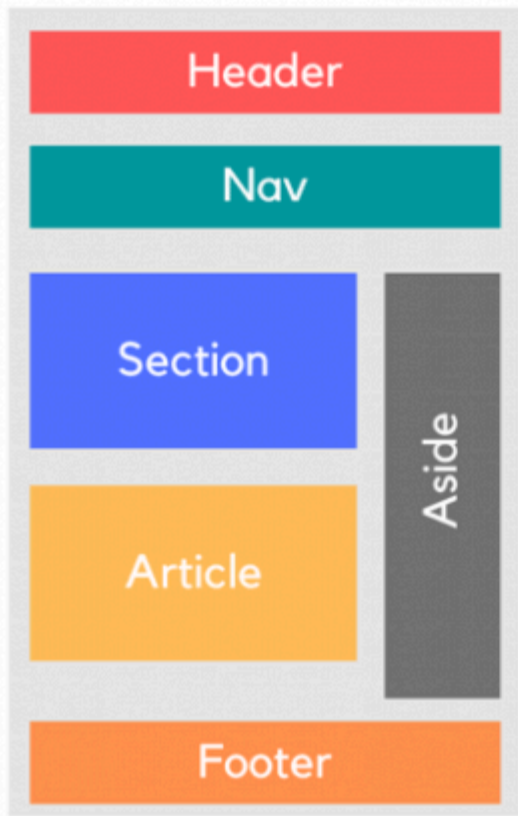


display: inline;

Inline elements flow from one line to the next.

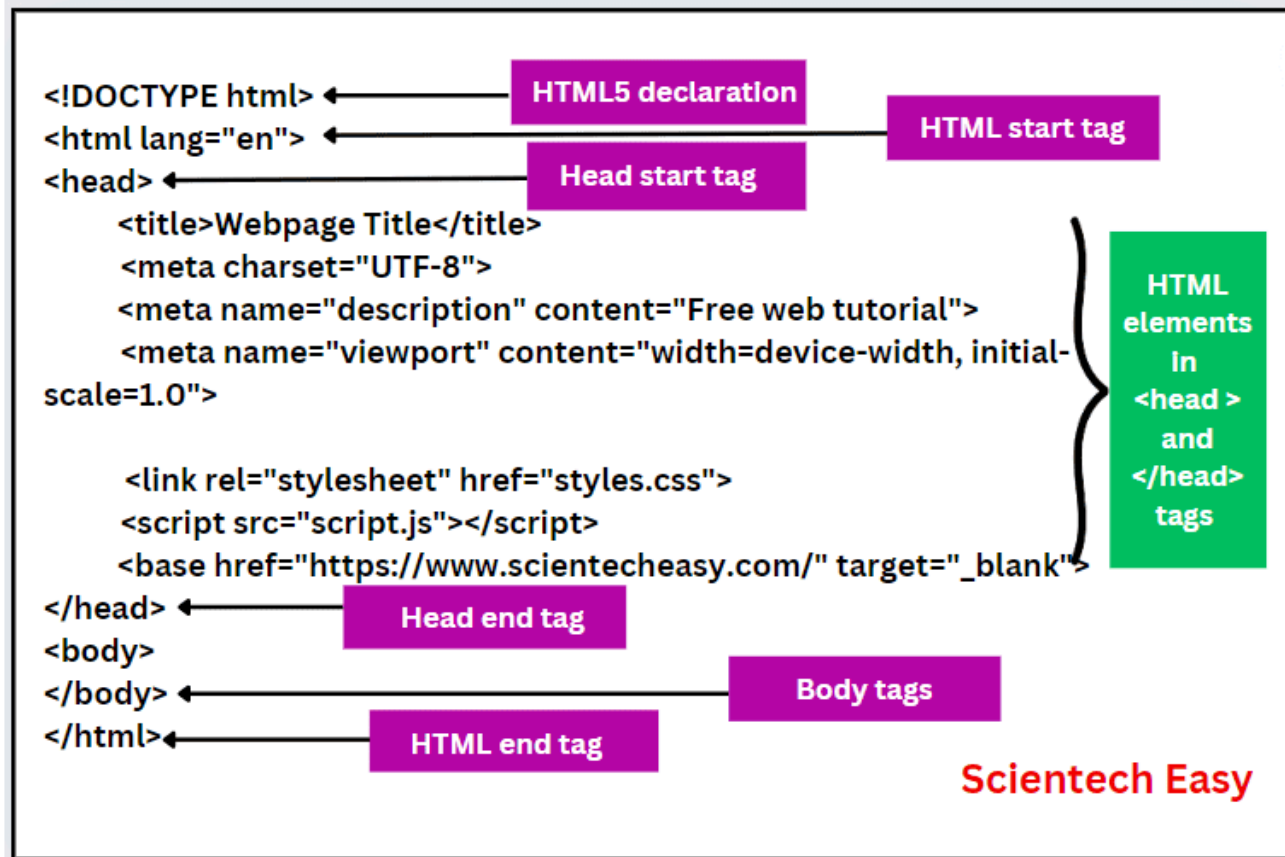
Elementos de HTML





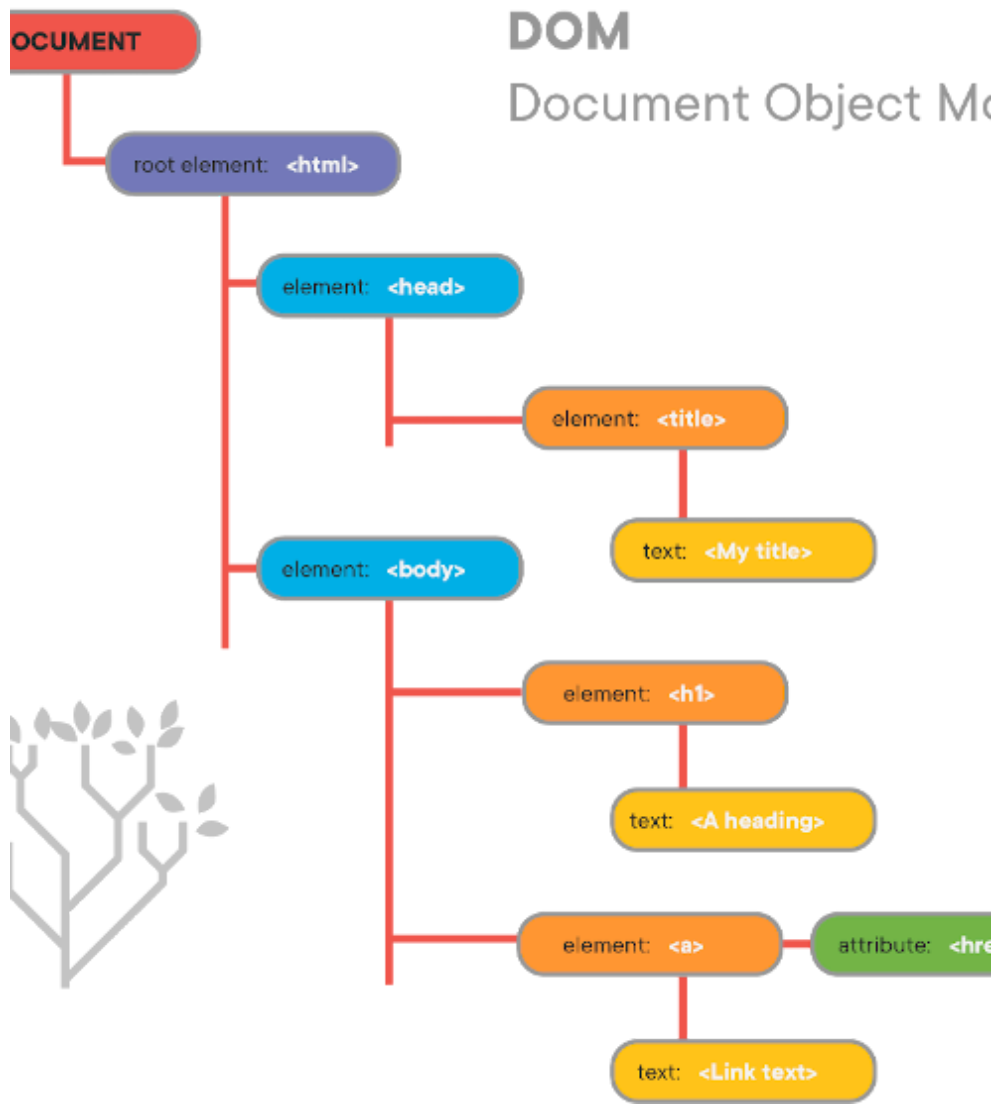
Semantic Elements

A Complete Guide to HTML Header



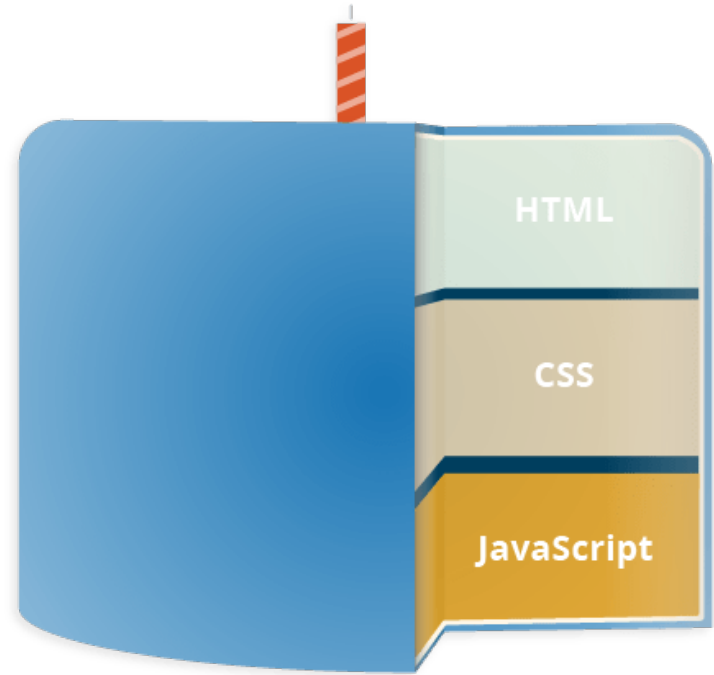
Document Object Model

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Title</title>
  </head>
  <body>
    <h1>A heading</h1>
    <a
      href="http://example.com">
      Link text</a>
  </body>
</html>
```

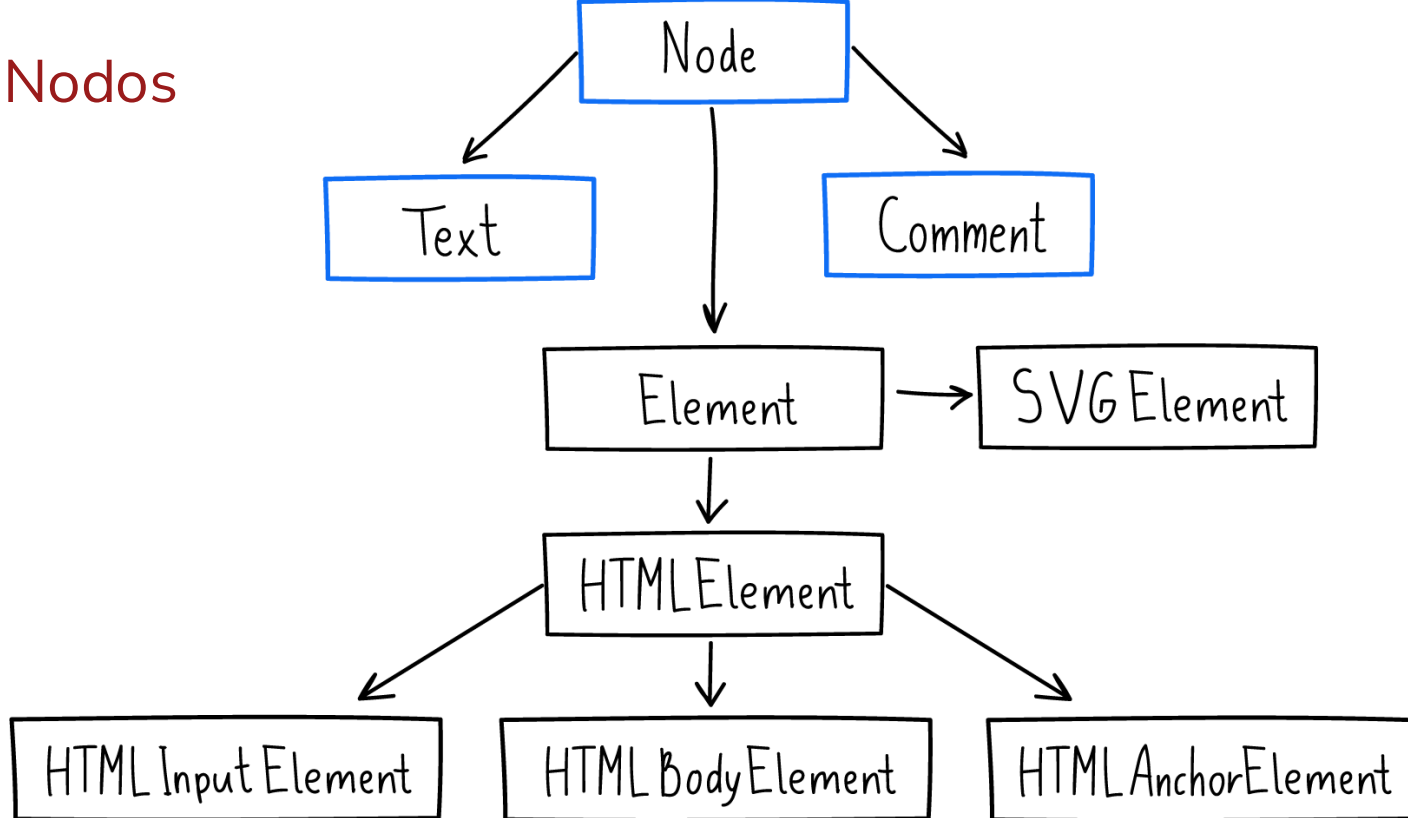


HTML y CSS se manipulan con
el DOM

 The JavaScript DOM explained in 5
minutes!



Tipos de Nodos



Browser Object Model también en los navegadores

Buscando los nodos

```
// element selectors = Methods used to target and manipulate HTML elements
//                      They allow you to select one or multiple HTML elements
//                      from the DOM (Document Object Model)

//      with the methods:

//      document.getElementById()           // ELEMENT OR NULL
//      document.getElementsByClassName()    // HTML COLLECTION
//      document.getElementsByTagName()      // HTML COLLECTION
//      document.querySelector()            // FIRST ELEMENT OR NULL
//      document.querySelectorAll()         // HTML COLLECTION
```



[Learn JavaScript ELEMENT SELECTORS easy!](#)

Recorriendo el DOM

```
// DOM Navigation = The process of navigating through the structure
//                  of an HTML document using JavaScript.

//            with the properties:

//            .firstElementChild
//            .lastElementChild
//            .nextElementSibling
//            .previousElementSibling
//            .parentElement
//            .children
```



[Learn DOM Navigation in 15 minutes!](#)

Cambiando el DOM

```
// STEP 1 CREATE THE ELEMENT
const newH1 = document.createElement("h1");

// STEP 2 ADD ATTRIBUTES/PROPERTIES
newH1.textContent = "I like pizza!";
newH1.id = "myH1";
newH1.style.color = "tomato";
newH1.style.textAlign = "center";

// STEP 3 APPEND ELEMENT TO DOM
document.body.append(newH1);

// or
document.body.innerHTML = '<h1 id="myH1" style="color:tomato; text-align:center;"> I like pizza </h1>'
```



How to ADD/CHANGE HTML using JavaScript

Browser events

```
// eventListener = Listen for specific events to create interactive web pages
//           events: click, mouseover, mouseout

//           .addEventListener(event, callback);

const myBox = document.getElementById("myBox");

myBox.addEventListener("click", event => {
  event.target.style.backgroundColor = "tomato";
  event.target.textContent = "OUCH! 🤕";
});
```



MOUSE EVENTS in 10 minutes!

fetch

```
// fetch = Function used for making HTTP requests to fetch resources.  
//      (JSON style data, images, files)  
//  
//      Simplifies asynchronous data fetching in JavaScript and  
//      used for interacting with APIs to retrieve and send  
//      data asynchronously over the web.  
//  
//      fetch(url, {options})  
//  
  
fetch('https://dogapi.dog/api/v1/facts?number=2')  
  .then(response => response.json())  
  .then(data => console.log(data.facts))
```



How to FETCH data from an API
